DQI Algorithm Walkthrough

Kai, Ben, Mary, Mihir | June 26, 2026

Setup and Goal

Problem setup

Consider the max-XORSAT problem

$$Bx = v, \qquad B \in \mathbb{F}_2^{m\times n}, \quad v \in \mathbb{F}_2^m,$$

and we want to find an x that best fits.

Goal

Prepare a quantum state of the form

$$\sum_x P(f(x)) \left|x\right\rangle ,$$

where the correct answer has a large coefficient, so it has a high probability of being measured.
The idea is to first prepare a different quantum state, and then convert it into the target state by applying a Hadamard transform.

Step 0: Choose the initial quantum state

Start with

$$\left|\psi_0\right\rangle = \left|0^m\right\rangle_C \otimes \left|0^n\right\rangle_S \otimes \left|0^p\right\rangle_D .$$

The three registers are:

- C — constraint register, used to store vectors y that select which constraints are activated.
- S — syndrome register, used to store the syndromes $B^T y$.
- D — decoder register.

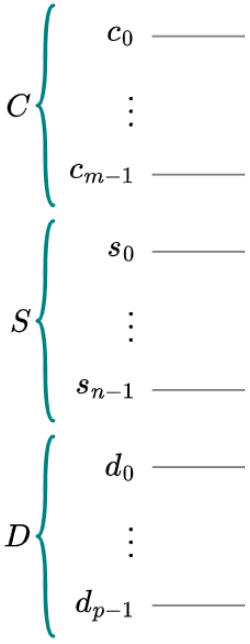


Figure 1: Initial register layout for $\left|\psi_0\right\rangle$.

Step 1: Prepare the weighted superposition of Dicke states

We prepare the C register as a weighted superposition of Dicke states.

Dicke states

$$|D_{m,j}\rangle = \frac{1}{\sqrt{\binom{m}{j}}} \sum_{\substack{|y|=j \\ y\in\{0,1\}^m}} |y\rangle\,.$$

We want to prepare

$$\sum_{j=0}^{\ell} \beta_j\,|D_{m,j}\rangle = \frac{1}{N}\sum_{j=0}^{\ell}\gamma_j \sum_{|y|=j}|y\rangle = \frac{1}{N}\sum_{|y|\le\ell}\gamma_{|y|}\,|y\rangle\,.$$

The coefficients are

$$\beta_j = \frac{\gamma_j\sqrt{\binom{m}{j}}}{N},$$

$$\gamma_j = \gamma_{|y|} = \sum_{k=0}^{\ell} c_k \sum_{\substack{\alpha_1+\cdots+\alpha_m=k \\ \alpha_i\ge 0 \\ \alpha \bmod 2 = y}} \frac{k!}{\alpha_1!\cdots\alpha_m!},$$

$$N = \sqrt{\sum_{j=0}^{\ell}\binom{m}{j}|\gamma_j|^2}.$$

Here, c_k comes from $P(x) = \sum_k c_k x^k$. The γ_j coefficients come from expanding $P(f(x))$, since

$$f(x) = \sum_{i=1}^{m}(-1)^{B_i\cdot x + v_i}.$$

There is a known algorithm (a set of gates) that prepares a weighted Dicke superposition with chosen coefficients, using $\mathcal{O}(m^2)$ gates.

State after Step 1

$$|\psi_1\rangle = \frac{1}{N}\sum_{|y|\le\ell}\gamma_{|y|}\,|y\rangle_C\,|0^n\rangle_S\,|0^p\rangle_D\,.$$

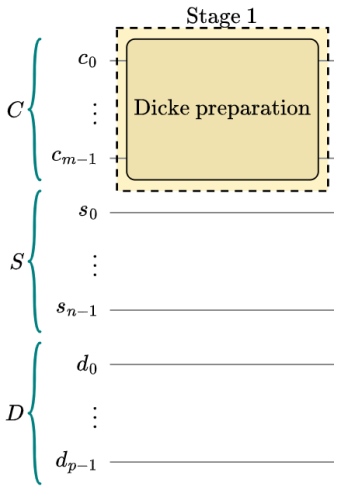


Figure 2: Preparation of the weighted Dicke superposition on the C register.

Step 2: Apply a phase onto each component

For one qubit,

$$Z|0\rangle = (-1)^0|0\rangle = |0\rangle\,, \qquad Z|1\rangle = (-1)^1|1\rangle = -|1\rangle\,.$$

Thus, apply Z gates according to the right-hand-side vector v. In each component $|y\rangle$, this produces the phase factor $(-1)^{v\cdot y}$.

State after Step 2

$$|\psi_2\rangle = \frac{1}{N}\sum_{|y|\le\ell}\gamma_{|y|}(-1)^{v\cdot y}\,|y\rangle_C\,|0^n\rangle_S\,|0^p\rangle_D\,.$$

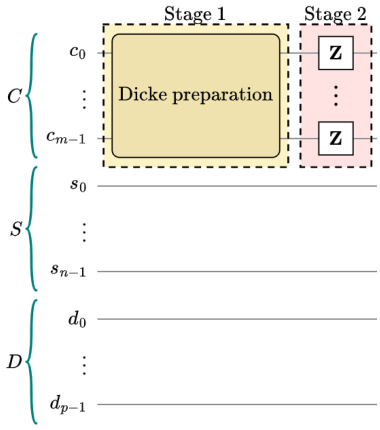


Figure 3: Phase application with Z gates on the constraint register.

Step 3: Set up the syndrome problems

Now we need to store $|B^T y\rangle$ on the S register.

To XOR in a quantum circuit, use the CNOT gate:

$$\mathrm{CNOT}\,|x\rangle\,|y\rangle = |x\rangle\,|x\oplus y\rangle\,.$$

Conceptually, CNOT flips the value of y if $x = 1$, and does nothing if $x = 0$.

Each qubit S_i in the syndrome register is

$$(B^T y)_i = B_{1,i}y_1 \oplus B_{2,i}y_2 \oplus \cdots \oplus B_{m,i}y_m.$$

Thus, the matrix entries $B_{r,i} \in \{0,1\}$ determine which y_r values get XORed together.

So, we CNOT C_r onto S_i whenever $B_{r,i} = 1$.

State after Step 3

$$|\psi_3\rangle = \frac{1}{N}\sum_{|y|\le\ell}\gamma_{|y|}(-1)^{v\cdot y}\,|y\rangle_C\,|B^T y\rangle_S\,|0^p\rangle_D\,.$$

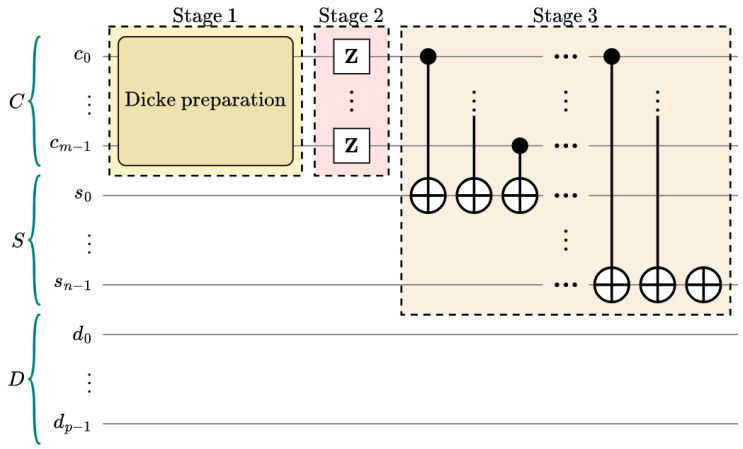


Figure 4: CNOT gates that computes the syndrome $B^T y$ into the S register.

Step 4: Solve the syndrome problems and uncompute the necessary registers

4.1: Decode $B^T y$ and store y on the D register

Implement the classical decoder (which can find a unique solution up to Hamming weight ℓ) as a quantum gate, such that

$$U_{\text{dec}}^{(\ell)} \left|B^T y\right\rangle_S |0^p\rangle_D = \left|B^T y\right\rangle_S |y\rangle_D .$$

Therefore,

$$|\psi_{4.1}\rangle = \frac{1}{N} \sum_{|y|\leq \ell} \gamma_{|y|} (-1)^{v\cdot y} |y\rangle_C \left|B^T y\right\rangle_S |y\rangle_D .$$

4.2: Clear the C and D registers

First, clear C using the decoded copy of y stored in D:

$$\text{CNOT} |y\rangle_D |y\rangle_C = |y\rangle_D |0^m\rangle_C .$$

Next, clear D by reversing the decoder with $(U_{\text{dec}}^{(\ell)})^\dagger$:

$$(U_{\text{dec}}^{(\ell)})^\dagger \left|B^T y\right\rangle_S |y\rangle_D = \left|B^T y\right\rangle_S |0^p\rangle_D .$$

State after Step 4

$$|\psi_{4.2}\rangle = \frac{1}{N} \sum_{|y|\leq \ell} \gamma_{|y|} (-1)^{v\cdot y} |0^m\rangle_C \left|B^T y\right\rangle_S |0^p\rangle_D .$$

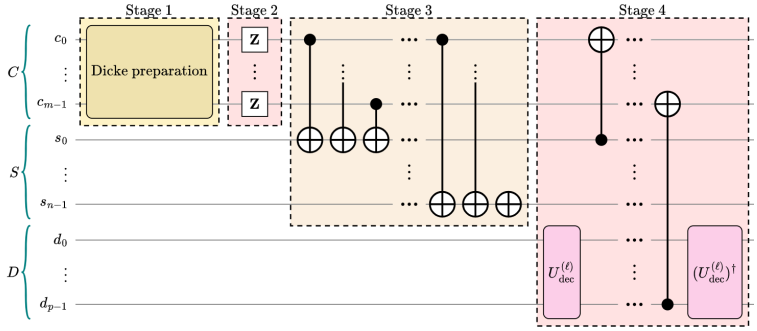


Figure 5: Decode $B^T y$ to recover y, then uncompute the extra registers.

Step 5: Apply Hadamard gates to the remaining S register

Since the C and D registers are all 0, we drop them from the notation: they provide no new information about the state and are no longer entangled with S.

Apply Hadamards to the syndrome register:

$$\begin{aligned} H^{\otimes n}\left|\psi_{4.2}\right\rangle &= \frac{1}{N} H^{\otimes n} \sum_{|y|\leq \ell} \gamma_{|y|} (-1)^{v\cdot y} \left|B^T y\right\rangle_S \\ &= \frac{1}{N\sqrt{2^n}} \sum_{x\in\{0,1\}^n} \left[\sum_{|y|\leq \ell} \gamma_{|y|} (-1)^{v\cdot y} (-1)^{B^T y\cdot x}\right] \left|x\right\rangle_S \\ &= \left|\psi_5\right\rangle = \frac{1}{N\sqrt{2^n}} \sum_{x\in\{0,1\}^n} P(f(x)) \left|x\right\rangle . \end{aligned}$$

This is the target form: the amplitudes are proportional to $P(f(x))$, so measuring the state favors solutions with larger coefficients.

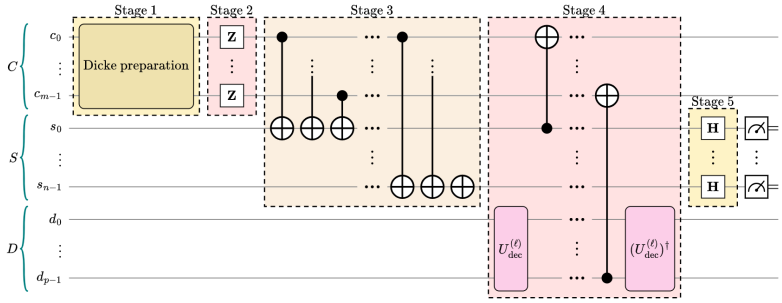


Figure 6: The full circuit, ending with Hadamard gates and measurement on the S register.

Appendix: Blocked Hadamard Decoder

Goal

Implement the decoder

$$U_{\text{dec}} = U_{\text{dec}}^{(R_1)} U_{\text{dec}}^{(R_2)} \cdots U_{\text{dec}}^{(R_k)}$$

as Step 4 of the DQI circuit. Each R_k is one block in the covering, and the current block will be denoted by R.

Input state for the decoder

We start with

$$\sum_y \alpha_y \left|y\right\rangle_C \left|B^T y\right\rangle_S \left|0^{w2^d}\right\rangle_T \left|0\right\rangle_a ,$$

where T is the lookup table register and a is an ancilla flag.

One-block overview

For one block R, the block decoder has the form

$$U_{\text{dec}}^{(R)} = U_1^\dagger U_2^\dagger U_3 U_2 U_1,$$

where the operations are applied in the order

$$\boxed{\text{build table} \longrightarrow \text{FWHT} \longrightarrow \text{clear } C \longrightarrow \text{undo FWHT} \longrightarrow \text{unbuild table}}.$$

Step	Operation	Purpose
1	Build $f(B, B^T y)$	Fill the lookup table T
2	Apply reversible FWHT	Convert f into F
3	Check entries $F_i = n$	Use the matching entry to XOR into C
2^{-1}	Undo reversible FWHT	Return F back to f
1^{-1}	Undo lookup-table build	Return T to 0^{w2^d}

Workflow for one block

The state begins as

$$\left|y\right\rangle \left|B^T y\right\rangle \left|0^{w2^d}\right\rangle \left|0\right\rangle .$$

$$\begin{aligned}\left|y\right\rangle \left|B^T y\right\rangle \left|0^{w2^d}\right\rangle \left|0\right\rangle &\xrightarrow{1} \left|y\right\rangle \left|B^T y\right\rangle \left|f(B, B^T y)\right\rangle \left|0\right\rangle \\ &\xrightarrow{2} \left|y\right\rangle \left|B^T y\right\rangle \left|F(B, B^T y)\right\rangle \left|0\right\rangle .\end{aligned}$$

Here,

$$\left|f(B, B^T y)\right\rangle = \left|f_{B,B^T y}(0 \cdots 0)\right\rangle \cdots \left|f_{B,B^T y}(1 \cdots 1)\right\rangle .$$

Sequentially, for each F_i,

$$\begin{aligned}\left|y\right\rangle \left|B^T y\right\rangle \left|F(B, B^T y)\right\rangle \left|0\right\rangle &\xrightarrow{3.1} \left|y\right\rangle \left|B^T y\right\rangle \left|F(B, B^T y)\right\rangle \left|[F_i = n]\right\rangle_a \\ &\xrightarrow{3.2} \left|y \oplus [F_i = n]\widetilde{z}_i\right\rangle \left|B^T y\right\rangle \left|F(B, B^T y)\right\rangle \left|[F_i = n]\right\rangle_a \\ &\xrightarrow{3.3} \left|y \oplus [F_i = n]\widetilde{z}_i\right\rangle \left|B^T y\right\rangle \left|F(B, B^T y)\right\rangle \left|0\right\rangle_a .\end{aligned}$$

By the end of Step 3,

$$\xrightarrow{3} \left|0^m\right\rangle \left|B^T y\right\rangle \left|F(B, B^T y)\right\rangle \left|0\right\rangle_a ,$$

since $[F_i = n]\widetilde{z}_i$ is only nonzero when

$$[F_i = n]\widetilde{z}_i = y.$$

Here, z_i is the input for F_i, and $\widetilde{z_i}$ is the m-bit bitstring corresponding to z_i.
Then uncompute the table workspace:

$$|0^m\rangle \left|B^T y\right\rangle \left|F(B, B^T y)\right\rangle |0\rangle \xleftarrow{2} |0^m\rangle \left|B^T y\right\rangle \left|f(B, B^T y)\right\rangle |0\rangle \\ \xleftarrow{1} |0^m\rangle \left|B^T y\right\rangle \left|0^{w2^d}\right\rangle |0\rangle .$$

Repeat for all blocks.

Step 1: Build the lookup table

For a given syndrome $s = B^T y$ and sub-block R of B^T, update the table by

$$|s_j\rangle \left|T_{R_j}\right\rangle \mapsto |s_j\rangle \left|T_{R_j} + (-1)^{s_j}\right\rangle .$$

Here, R_j is the j-th row of R.

Gate rule for one table update

For each j, use

$$X_{s_j}\,\mathrm{CINC}_{s_j}(T_{R_j})\,X_{s_j}\,\mathrm{CDEC}_{s_j}(T_{R_j}).$$

This adds 1 if $s_j = 0$, and subtracts 1 otherwise.

Each register value is in $[-n, n]$, so allocate

$$q = \lceil \log_2(2n+1) \rceil$$

qubits per counter.

Step 2: Apply the reversible Hadamard transform to the table

Arithmetic on binary bits is defined modulo Q. Pick

$$Q > 2n$$

to uniquely represent every possible value in $[-n, n]$.

Reversible FWHT butterfly

To implement

$$(A, B) \mapsto (A + B, A - B),$$

use the following arithmetic updates:

$$A \mapsto A + B \pmod{Q},$$
$$B \mapsto -2B \pmod{Q},$$
$$B \mapsto B + A \pmod{Q}.$$

Equivalently, using modular arithmetic gates,

$$\mathrm{MODADD}_Q(B, A)\,\mathrm{MULCONST}_Q(B, -2)\,\mathrm{MODADD}_Q(A, B),$$

where gates are written right-to-left.

Use the same scheduler code for FWHT to build the quantum circuit.

Step 3: Clear the C register

For each

$$z_i \in \{0,1\}^d, |z_i| \le \ell$$

which is represented on T_{z_i}, we first check ownership.

Ownership dictionary

Since $\widetilde{z_i}$'s repeat across blocks, we say $\widetilde{z_i}$ with $|z_i| \leq \ell$ is owned by a block R_k. Construct a classical dictionary as follows: loop through the subsets of R_1 and assign any new subset (with weight up to ℓ) to R_1. Then repeat for $R_2, R_3, \ldots, R_k$. This also avoids recovering candidates with weight higher than ℓ.

Substep	Action
3.1	Set the flag if $T_{z_i} = n$
3.2	If owned, XOR $\widetilde{z_i}$ into C using the flag
3.3	Apply the equality check again to clear the flag

3.1: Update the flag

The goal is: if $T_{z_i} = n$, set

$$|0\rangle_a \mapsto |1\rangle_a .$$

If the j-th bit of the binary representation of n is 0, apply X onto $(T_{z_i})_j$. This uniquely sends the bitstring $T_{z_i} = n$ to

$$|1 \cdots 1\rangle .$$

Then apply a multi-controlled CNOT onto the ancilla:

$$\mathrm{MCX}(T_{z_i}, a).$$

Finally, undo the X gates on $(T_{z_i})_j$. Call this whole comparison circuit

$$\mathrm{EQ}_n(T_{z_i}, a).$$

3.2: XOR onto C

Let

$$\vec{r} = \{r_1, \ldots, r_d\}$$

define which columns were kept in the current cover. For example,

$$\vec{r} = \{1, 3, 4\}.$$

If $(z_i)_j = 1$ and z_i is owned by the current block R according to the dictionary lookup, apply a controlled X by the ancilla onto c_{r_j}:

$$\mathrm{CX}(a, c_{r_j}).$$

3.3: Clear the flag

Apply the same equality check again:

$$\mathrm{EQ}_n(T_{z_i}, a).$$

This returns the ancilla flag to $|0\rangle_a$.

Undo Step 2

Undo the reversible Hadamard transform by mapping

$$(A + B, A - B) \mapsto (A, B).$$

Inverse FWHT butterfly

Use

$$B \mapsto B - A,$$

$$B \mapsto (-2)^{-1}B,$$
$$A \mapsto A - B.$$

In modular gate form,

$$\mathrm{MODSUB}_Q(A,B)\,\mathrm{MULCONST}_Q(B,(-2)^{-1})\,\mathrm{MODSUB}_Q(B,A),$$

where gates are written right-to-left.

Use the same scheduler code.

Undo Step 1

For each j, undo the lookup-table update with

$$\mathrm{CINC}_{s_j}(T_{R_j})\,X_{s_j}\,\mathrm{CDEC}_{s_j}(T_{R_j})\,X_{s_j}.$$

This reverses the update

$$T_{R_j} \mapsto T_{R_j} + (-1)^{s_j}.$$

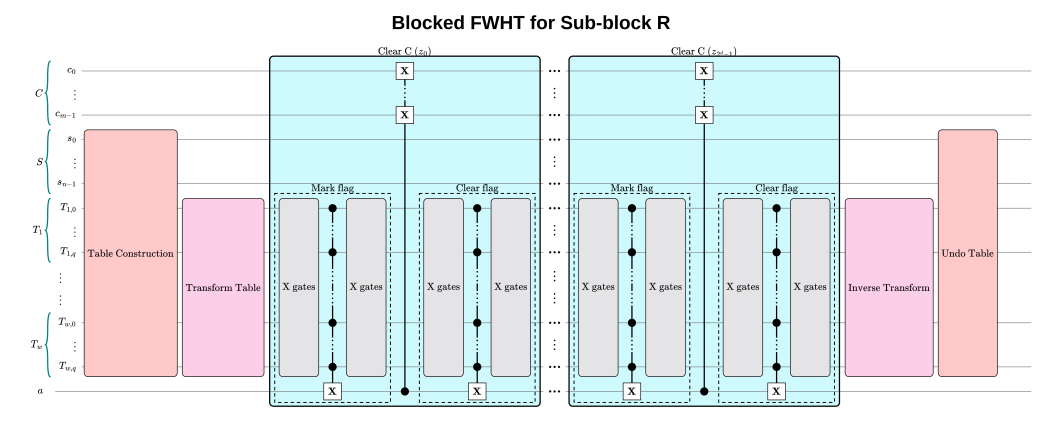


Figure 7: Blocked FWHT decoder circuit for a sub-block R.

Constructing the full decoder

To construct

$$U_{\mathrm{dec}} = U_{\mathrm{dec}}^{(R_1)} U_{\mathrm{dec}}^{(R_2)} \cdots U_{\mathrm{dec}}^{(R_k)},$$

we only need to pass in the current block R to construct each block decoder. The ownership dictionary is useful to pass in beforehand, to avoid computing it inside the algorithm.

Additional Notes

Modular arithmetic can be implemented efficiently using Mersenne-modulus circuits; pick k such that

$$Q = 2^k - 1 > 2n.$$

Table construction in Step 1 can be implemented more efficiently as follows. Since

$$(-1)^{s_j} = 1 - 2s_j,$$

rewrite

$$T_u = \sum_{j:R_j=u} (-1)^{s_j}$$

as

$$T_u = c_u - 2 \sum_{j:R_j=u} s_j \pmod{Q},$$

where

$$c_u = |\{j : R_j = u\}|$$

is the row multiplicity. Therefore, we apply

$$T_u \to T_u + c_u \qquad \text{and} \qquad T_{r_j} \to T_{r_j} - 2s_j$$

for all j. This can be implemented with

$$\text{ADD_CONST}_Q(c_u, T_u) \prod_{j:R_j=u} \text{CADD}_Q^{s_j}(-2, T_u).$$